Contagem distribuída de produtos em imagens: coordenação por web services e relógios lógicos de Lamport

André Vieira Penchel, **Matheus Campello**, **João Vitor**, **Henrique Saldanha**, **Mateus Martins Parreiras** e **Arthur Campos** [**PUC Minas**]

Instituto de Ciências Exatas e Informática, Pontifícia Universidade Católica de Minas Gerais, Belo Horizonte, MG, Brasil. Disciplina de Computação Distribuída, Prof. Luigi D. C. Soares. Primeira entrega.

Resumo. Este trabalho implementa um sistema distribuído que conta quantas unidades de um produto, descrito em linguagem natural ("garrafa de água"), aparecem em uma fotografia de prateleira ou estoque. A imagem é segmentada em superpixels com SLIC e cada superpixel é classificado pelo modelo CLIP, etapa cara em CPU. Uma API coordenadora corta a imagem em blocos sobrepostos, distribui os blocos por HTTP a dois ou mais workers que executam a segmentação e a classificação, costura as máscaras devolvidas e conta as instâncias. A comunicação entre os nós usa web services REST e todas as mensagens carregam carimbos de relógio lógico de Lamport, implementado manualmente, que permitem ordenar os eventos de envio e recepção entre máquinas sem depender de relógios físicos. O sistema foi executado em três máquinas virtuais na AWS provisionadas por Terraform, e o código, os testes e as instruções de execução estão em um repositório público no GitHub.

Palavras-chave: Sistemas distribuídos, web services, relógio lógico de Lamport, segmentação de imagens, SLIC, CLIP

1 O problema escolhido

Escolhemos a sugestão de tema 3 do enunciado, segmentação de imagens distribuída com SLIC, aplicada a um problema concreto: dado o nome de um produto em português e uma foto de prateleira, armário ou estoque, responder quantas unidades daquele produto aparecem na foto. O usuário não precisa treinar nada nem cadastrar o produto antes; basta escrever o nome.

O algoritmo em uma única máquina funciona em quatro etapas. Primeiro a imagem é reduzida a no máximo 1000 pixels de lado e dividida em cerca de 600 superpixels com o SLIC [Achanta *et al*., 2012], usando a implementação do scikit-image [van der Walt *et al*., 2014]. Em seguida, cada superpixel é recortado com uma janela de contexto e pontuado pelo CLIP [Radford *et al*., 2021], um modelo que compara imagens e textos no mesmo espaço vetorial: frases como "a photo of a water bottle on a store shelf" são comparadas com as mesmas frases montadas para uma lista de negativos ("wooden shelf", "price tag", "cardboard box", entre outros), e o superpixel recebe a probabilidade de ser o objeto procurado. Os superpixels acima de um limiar formam uma máscara binária, que passa por operações morfológicas. Por fim, a máscara é separada em instâncias com transformada de distância e *watershed*, e as instâncias com área mínima são contadas e devolvidas com suas caixas delimitadoras.

A etapa do CLIP domina o custo: são centenas de inferências de uma rede neural por imagem, todas em CPU. Esse é o trabalho que dividimos entre máquinas. Como cada superpixel é classificado de forma independente, a imagem pode ser cortada em blocos e cada bloco processado por um nó diferente, o que caracteriza o problema como "divisível em várias unidades de trabalho executadas de maneira distribuída", como pede o enunciado.

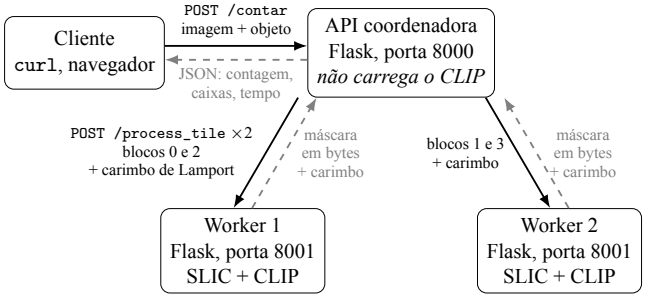


Figura 1. Entidades e mensagens. A API corta a imagem em 4 blocos, envia dois para cada worker em paralelo e costura as máscaras devolvidas. Toda mensagem leva um carimbo de Lamport.

2 Arquitetura do sistema

O sistema tem três tipos de entidade, ilustrados na Figura 1.

Cliente. Qualquer programa capaz de fazer uma requisição HTTP: usamos `curl` nos testes, mas a API libera CORS para ser chamada de um navegador. O cliente envia a foto e o nome do produto e recebe um JSON.

API coordenadora. É o nó que conhece o problema inteiro, mas não executa a parte pesada: ela não carrega o modelo CLIP. Ao receber `POST /contar`, ela valida os parâmetros, reduz a imagem, corta-a em uma grade 2×2 com 20% de sobreposição entre blocos vizinhos (para que um produto na fronteira não seja cortado ao meio), e despacha os blocos em paralelo, alternando entre os workers: blocos 0 e 2 vão para o primeiro, 1 e 3 para o segundo. Cada bloco recebe uma quantidade de superpixels proporcional à sua área, para que o tamanho médio de um superpixel seja o mesmo da versão em uma máquina. Quando as quatro máscaras voltam, a API copia apenas a zona central de cada uma para uma máscara global do tamanho da imagem, descartando as margens sobrepostas, e executa a separação e a contagem de instâncias localmente, pois essa etapa é barata e precisa da imagem in-

Tabela 1. Mensagens trocadas entre as entidades.

Mensagem	Conteúdo
Cliente → API `POST /contar`	multipart: `imagem` (arquivo), `objeto` (texto) e parâmetros opcionais (`limiar`, `segmentos`, …).
API → Cliente	JSON: `quantidade`, lista de `instancias` com caixa, centro e área em pixels da imagem original, `tempo_ms`.
API → Worker `POST /process_tile`	multipart: bloco em JPEG, `objeto`, `limiar`, `segmentos`, `compacidade`, `janela`; cabeçalhos `X-Task-Id` e `X-Lamport-Timestamp`.
Worker → API	máscara binária em bytes (`application/octet-stream`); cabeçalhos `X-Task-Id`, `X-Lamport-Timestamp` e `X-Worker-Id`.
API → Worker `GET /health`	sem corpo; a API consolida o estado de cada worker no seu próprio `GET /health` (`ok` ou `degradado`).

teira. Se qualquer bloco falhar, a resposta é HTTP 503 com a lista dos blocos que falharam; nunca devolvemos uma contagem parcial, que seria um resultado errado disfarçado de certo.

Workers. São processos idênticos e sem estado, cada um em uma máquina, que carregam o CLIP uma única vez ao subir e atendem `POST /process_tile`. Um worker não sabe de que imagem o bloco veio nem quantos workers existem; ele só segmenta, classifica e devolve a máscara.

A Tabela 1 resume quem fala com quem e o conteúdo de cada mensagem. A máscara é devolvida como bytes crus (um byte 0 ou 1 por pixel, tipo `application/octet-stream`) em vez de JSON: para um bloco de 600×600 pixels isso são 360 KB, contra vários megabytes se fosse uma lista JSON.

3 Requisitos implementados

Dos requisitos listados para a primeira entrega, implementamos dois: utilização de web services e sincronização de relógios lógicos.

3.1 Web services

Toda a comunicação entre os nós é feita por HTTP, no estilo REST, com o microframework Flask no lado servidor e a biblioteca `requests` no lado cliente. O enunciado permite bibliotecas para a comunicação, então o que é "de prateleira" aqui é o transporte HTTP; o protocolo de aplicação foi definido por nós: as rotas, os campos de cada mensagem, os cabeçalhos de identificação de tarefa e de relógio, a regra de despacho alternado, a sobreposição dos blocos e a costura das máscaras.

Escolhemos web services em vez de gRPC por três motivos. O primeiro é a facilidade de teste e demonstração: qualquer rota pode ser exercitada com `curl` ou de um navegador, sem gerar código a partir de um arquivo `.proto`. O segundo é que o produto final é uma API que um front-end chamaria diretamente, então o cliente já falaria HTTP de qualquer forma. O terceiro é a simplicidade de depuração: as mensagens entre API e workers são legíveis nos logs de cada máquina.

A API expõe quatro rotas. `POST /contar` é a principal e devolve o JSON descrito na Tabela 1. `POST /contar/figura` recebe a mesma entrada e devolve um PNG com a foto, a máscara costurada e as instâncias numeradas, útil para conferir o resultado visualmente. `GET /objetos` lista os nomes em português que a API traduz para inglês antes de montar as frases do CLIP. `GET /health` consulta o `/health` de cada worker e responde `ok` ou `degradado`. O worker expõe `POST /process_tile` e `GET /health`. Entradas inválidas (imagem vazia, limiar fora de $[0, 1]$, arquivo acima de 20 MB) são recusadas com 400 ou 413 antes de qualquer envio aos workers, e o worker recusa com 422 uma requisição sem os cabeçalhos obrigatórios.

Os quatro blocos são enviados ao mesmo tempo, com um *pool* de threads na API, e cada envio tem um tempo limite configurável (padrão de 300 s). Como os workers são sem estado, o endereço deles é apenas uma lista na variável de ambiente `WORKER_URLS`; adicionar uma terceira máquina é acrescentar uma URL.

3.2 Relógio lógico de Lamport

Cada nó mantém um relógio lógico de Lamport [Lamport, 1978]: um contador inteiro que começa em zero. Implementamos as duas regras manualmente, sem biblioteca, em um módulo de 25 linhas protegido por uma trava, já que a API e os workers atendem várias requisições em threads concorrentes:

- **Evento local ou envio:** `tick()` incrementa o contador e devolve o novo valor, que vai no cabeçalho `X-Lamport-Timestamp` da mensagem.
- **Recepção:** `receive(remoto)` faz $C \leftarrow \max(C, C_{remoto}) + 1$.

Na API, cada bloco despachado é um envio (`tick`) e cada resposta recebida, inclusive de erro, é uma recepção. No worker, a chegada de um bloco é uma recepção e a resposta é um envio; o carimbo do worker volta no cabeçalho da resposta, de modo que a API avança seu relógio ao receber. O identificador da tarefa (`X-Task-Id`, formado por um identificador do pedido mais o índice do bloco) acompanha o carimbo para que os eventos de uma mesma requisição possam ser correlacionados nos logs das três máquinas. O trecho abaixo, abreviado, é do log real do primeiro teste na AWS, com a API em uma máquina e o worker 1 em outra:

```
[api]     SEND    task=t0 worker=172.31.10.180 lamport=1
[api]     SEND    task=t1 worker=172.31.11.218 lamport=2
[api]     SEND    task=t2 worker=172.31.10.180 lamport=3
[api]     SEND    task=t3 worker=172.31.11.218 lamport=4
[worker1] RECEIVE task=t0 remote=1 lamport=2
[worker1] RECEIVE task=t2 remote=3 lamport=4
[worker1] SEND    task=t0 lamport=13
```

O worker 1 recebeu `t0` com carimbo remoto 1 e passou a 2; recebeu `t2` com carimbo 3 e passou a 4, mesmo que seu próprio relógio estivesse atrás (o salto até 13 vem de linhas omitidas, em que ele recebeu os blocos de um segundo pedido). Sem consultar os relógios físicos das duas máquinas, a cadeia de mensagens mostra que o envio de `t0` pela API (1) precede sua recepção no worker (2), que precede a resposta (13). Os carimbos sozinhos garantem apenas a relação inversa: um evento de carimbo maior nunca precede

causalmente um de carimbo menor, logo a resposta não pode ter precedido nenhum dos quatro envios. Carimbo menor não implica "aconteceu antes": o envio de `t1` na API e a recepção de `t0` no worker 1 têm o mesmo carimbo 2 e são concorrentes, assim como os envios ao worker 2 são concorrentes com o processamento no worker 1. O relógio de Lamport dá só uma ordem parcial, e isso é esperado. O que o sistema ganha é uma forma de ordenar causalmente os eventos de cada requisição nos logs, detectar uma resposta que chegue com carimbo menor do que o esperado e, na segunda entrega, basear nele o tratamento de respostas atrasadas e reenvios.

4 Execução e evidências

O código está no repositório público do grupo no GitHub[1], ramo `main`; este relatório descreve o *commit* `5cc3b8f`, de 1º de novembro de 2026. O `README` traz as instruções de execução, e há 12 testes unitários (`python -m unittest`) que exercitam a API, o despacho, a costura e o relógio lógico sem carregar o modelo, usando workers falsos.

Em uma máquina, para desenvolvimento. Três terminais na raiz do repositório, com as dependências instaladas:

```
PORT=8001 WORKER_ID=worker1 python -m distribuido.worker
PORT=8002 WORKER_ID=worker2 python -m distribuido.worker
python api.py
```

A API procura os workers em `localhost`, portas 8001 e 8002, por padrão; em máquinas da mesma rede local basta apontar `WORKER_URLS` para os IPs delas.

Em máquinas separadas, na AWS. Para cumprir o requisito de nós em máquinas distintas, escrevemos a infraestrutura como código na pasta `infra/`, com Terraform. Um `terraform apply` cria cinco recursos na região `us-east-2`: o par de chaves SSH, um grupo de segurança que libera a porta 8000 apenas para o IP do grupo e a porta 8001 apenas entre as máquinas do próprio grupo, duas instâncias `t3.medium` (2 vCPUs, 4 GB) para os workers e uma `t3.small` para a API, todas com Ubuntu 24.04. Um *script* de inicialização, executado pela própria AWS no primeiro boot, clona o repositório, instala as dependências e registra um serviço do sistema que sobe o worker ou a API com as variáveis de ambiente certas; o Terraform preenche `WORKER_URLS` da API com os IPs internos dos workers. Nada é configurado manualmente nas máquinas.

Na execução de 1º de novembro de 2026, as máquinas receberam os endereços 18.119.119.159 (API), 3.15.11.21 e 18.119.115.239 (workers; 172.31.10.180 e 172.31.11.218 na rede interna). O `/health` consultado do computador do grupo respondeu:

```
{"status":"ok","workers":{"http://172.31.10.180:8001":true,
                          "http://172.31.11.218:8001":true}}
```

e a contagem de "garrafa de agua" na imagem `imagens/images.jpg` do repositório, enviada por `curl`, devolveu 4 instâncias com suas caixas em 38,6 s (`tempo_ms` de 38633), com os quatro blocos processados por dois workers em máquinas diferentes. O custo das três máquinas é de cerca de US$ 0,12 por hora, e elas são destruídas com `terraform destroy` ao fim de cada sessão de testes, então os endereços acima não permanecem ativos; a execução pode ser repetida com qualquer conta da AWS seguindo o `README`.

[1] `https://github.com/Matheuscamp29/Trabalho-Computacao-Distribuida`

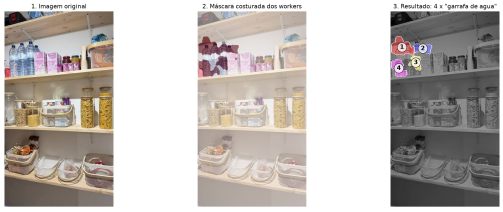


Figura 2. Saída de `POST /contar/figura` na AWS para "garrafa de agua": foto original, máscara costurada a partir dos quatro blocos e instâncias numeradas.

Como verificar. Com o sistema no ar, em qualquer das formas, três comandos reproduzem as evidências deste relatório, trocando `ENDERECO` pelo da API:

```
curl http://ENDERECO:8000/health
curl -F "imagem=@imagens/images.jpg" \
     -F "objeto=garrafa de agua" http://ENDERECO:8000/contar
ssh ubuntu@IP_DO_WORKER 'journalctl -u contador -f'
```

O terceiro mostra, ao vivo, as linhas `RECEIVE` e `SEND` com os carimbos de Lamport no worker enquanto o segundo é executado; em uma execução local, as mesmas linhas aparecem no terminal de cada processo.

5 Desafios e problemas encontrados

Contexto perdido nas bordas. O CLIP decide olhando o entorno de cada superpixel. Cortar a imagem em blocos sem sobreposição fazia produtos na fronteira serem classificados com metade do contexto e, muitas vezes, perdidos. A sobreposição de 20% e a costura apenas da zona central resolveram o caso da borda, mas um bloco ainda vê menos imagem do que a versão em uma máquina, e a contagem distribuída pode diferir da local na mesma foto. Esse é o principal ponto a investigar na análise de desempenho da segunda entrega.

Trabalho repetido por bloco. Na primeira versão a API repassava `segmentos=600` a cada bloco, o que gerava quatro vezes mais superpixels, e portanto quatro vezes mais inferências do CLIP, do que a versão local. Em um *notebook* isso passava despercebido; nas instâncias `t3.medium` cada bloco levava mais de 150 s e a API, que esperava 60 s, respondia 503 em todos os pedidos. Diagnosticamos pelos logs das duas máquinas (o worker registrava a resposta com código 200 mais de um minuto depois de a API ter desistido) e corrigimos distribuindo os superpixels proporcionalmente à área: o tempo por imagem caiu para 38,6 s.

Simplicidade do código. A primeira implementação da API usava recursos que dificultavam a leitura e a apresentação (decoradores de ciclo de vida, geradores, classes para configuração). Reescrevemos tudo em Flask com funções

simples, uma rota por função e o relógio lógico como um módulo de estado global protegido por trava. O comportamento foi conferido comparando os resultados numéricos das duas versões nas imagens de exemplo.

Ambiente de nuvem. A conta da AWS usada pelo grupo é do novo modelo de cadastro da AWS, que fixa a região do projeto por uma política da organização. O primeiro `terraform plan` em `us-east-1` falhou com "explicit deny in a service control policy" mesmo com o usuário tendo permissão de administrador; a causa só ficou clara na documentação da AWS, e a solução foi parametrizar a região e usar `us-east-2`.

6 Papel de cada aluno

- **Matheus Campello:** pipeline de visão em uma máquina (14/10): segmentação SLIC, pontuação dos superpixels com o CLIP e as frases positivas e negativas, máscara com operações morfológicas, separação de instâncias por *watershed* e contagem, tradução dos nomes de produto, figura de saída e imagens de exemplo.
- **Mateus Martins Parreiras:** primeira versão da execução distribuída (19/10): corte da imagem em blocos com sobreposição, gerente que despacha os blocos e costura as máscaras, e o processo worker.
- **Arthur Campos:** relógio lógico de Lamport (19/10): módulo do relógio, cabeçalhos `X-Task-Id` e `X-Lamport-Timestamp`, logs de envio e recepção, configuração com dois workers e os testes unitários do despacho, do relógio e do protocolo do worker.
- **João Vitor:** rotas da API (`/contar`, `/contar/figura`, `/objetos` e `/health`), validação das entradas com os códigos de erro e documentação de execução no `README`.
- **Henrique Saldanha:** infraestrutura na AWS: Terraform em `infra/`, *script* de inicialização das máquinas com o serviço do sistema, subida e teste das três instâncias.
- **André Vieira Penchel:** reescrita da API e dos workers em Flask com o código organizado em pastas (30/10), diagnóstico e correção do tempo limite e dos superpixels por bloco na AWS (01/11), coleta das evidências e redação do relatório.

7 Conclusão e próximos passos

O sistema cumpre os requisitos da primeira entrega com dois nós de processamento em máquinas distintas, comunicação por web services e relógios lógicos de Lamport implementados manualmente, e foi executado de ponta a ponta na AWS. Para a segunda entrega, o tratamento de falhas vai além do tempo limite atual: reenvio de um bloco ao outro worker quando o primeiro falha, verificação de saúde antes do despacho e tolerância à queda de um worker durante o processamento. A análise de desempenho comparará, no mesmo hardware, a execução em uma máquina com a distribuída, medindo tempo e diferença de contagem para as mesmas fotos.

Referências

Achanta, R., Shaji, A., Smith, K., Lucchi, A., Fua, P., and Süsstrunk, S. (2012). SLIC superpixels compared to state-of-the-art superpixel methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 34(11):2274–2282. DOI: 10.1109/TPAMI.2012.120.

Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565. DOI: 10.1145/359545.359563.

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. (2021). Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning*, pages 8748–8763.

van der Walt, S., Schönberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager, N., Gouillart, E., and Yu, T. (2014). scikit-image: image processing in Python. *PeerJ*, 2:e453. DOI: 10.7717/peerj.453.